

3. Posterior predictive and prior predictive sampling

Frank Edwards

Sampling from the posterior

- We use Bayesian methods to *approximate* the posterior distribution
- We can then sample parameters from the estimated posterior to learn about parameters or to simulate predictions
- We can plot densities, directly sample *credible intervals*, compare differences in means, all without relying on a theoretical *sampling distribution*, like we do with frequentist methods
- We can even simulate *new* data from the posterior

Quadratic approximation

- . Assume our posterior is approximately Normal (often reasonable)
- . Find the mode
- . Estimate curvature of the log posterior with a parabola
- . Use the resulting Gaussian distribution for inference
- . With enough data: quap $\rightarrow$ MLE

Quadrating approximation logic

Using quadratic approximation in R

9 draws from the bag of red and green marbles, we observe 6 green.

```
1 ### First, define the data as a list() object
2 ### let's treat 1 as green, 0 as red
3 dat<-list(G = 6, R = 3)
```

Using quadratic approximation in R

```
1 ## Specify the prior and likelihood in alist() object
2 formula_list<-alist(
3   # likelihood
4   G ~ dbinom(R+G, p),
5   # prior
6   p ~ dunif(0,1)
7 )
```

Using quadratic approximation in R

```
1 model1<-quap(  
2   formula_list,  
3   data = dat  
4 )
```

Using quadratic approximation in R

```
1 # summarize the distribution using summary() or precis()  
2 summary(model1)
```

```
      mean      sd    5.5%    94.5%  
p 0.6666667 0.1571338 0.4155365 0.9177968
```

Extract posterior samples

```
1 post_samples<-extract.samples(model1, n=10000)
```

Visualize

```
1 p1<-ggplot(post_samples, aes(y = p, x = 1:nrow(post_samples))) +  
2   geom_point(alpha = 0.1)  
3 p2<-ggplot(post_samples, aes(x = p)) +  
4   geom_density()  
5 ### grid.arrange() lets us put more than one plot in a window  
6 grid.arrange(p1, p2, ncol = 2)
```

Posterior prediction

Simulating prediction

- We've learned how to describe the parameter p with posterior samples.
- What would happen if we re-ran the experiment?
- **Posterior predictive simulation** incorporates updated information on the parameter into predictions of new values of our observed outcome variable (W , L)

Posterior prediction algorithm

- . Sample from the posterior distribution
- . Use these sampled posterior parameter values to sample new values from the likelihood

Simulating from one p

```
1 t <- extract.samples(model1, n=2)
2 t
```

```
      p
1 0.6589054
2 0.7941642
```

```
1 predicted_p1 <- rbinom(n=1, size = 9, prob = t$p[1])
2 predicted_p1
```

```
[1] 4
```

The distribution of predictions

```
1 predicted_p1 <- rbinom(n=10000, size = 9, prob = t$p[1])  
2 hist(predicted_p1, breaks = 0:9, main = t$p[1])
```

Simulating from a different value of p

```
1 predicted_p2 <- rbinom(n=10000, size = 9, prob = t$p[2])  
2 hist(predicted_p2, breaks = 0:9, main = t$p[2])
```

Averaging uncertainty

```
1 predicted_joint <- c(predicted_p1, predicted_p2)
2 hist(predicted_joint, breaks = 0:9, main = t)
```

Simulating over both values of p

```
1 predicted_p_joint <- rbinom(n=20000, size = 9, prob = t$p)
2 hist(predicted_p_joint, breaks = 0:9, main = t)
```

p = posterior median

```
1 p_samples <- extract.samples(model1, n = 10000)
2 ## set p at the median of the posterior
3 samples<-rbinom(1e4, 9, median(p_samples$p))
4 hist(samples, breaks = 0:9, main = median(p_samples$p))
```

p = posterior 5th percentile

```
1 ## set p at the 5th percentile of the posterior
2 samples<-rbinom(1e4, 9, quantile(p_samples$p, 0.05))
3 hist(samples, breaks = 0:9, main = quantile(p_samples$p, 0.05))
```

p = posterior 95th percentile

```
1 ## set p at the 95th percentile of the posterior
2 samples<-rbinom(1e4, 9, quantile(p_samples$p, 0.95))
3 hist(samples, breaks = 0:9, main = quantile(p_samples$p, 0.95))
```

Averaging over uncertainty in p

```
1 p_samples <- extract.samples(model1, n = 10000)
2 hist(p_samples$p)
```

Oops, impossible values!

```
1 # correct for impossible values
2 p_samples <- p_samples |>
3   mutate(p = case_when(
4     p > 1 ~ 1,
5     p < 0 ~ 0,
6     .default = p
7   ))
8
9 ggplot(p_samples, aes(x=p)) + geom_histogram()
```

Generate a prediction of the data for each p

```
1 RG_predictions<-rbinom(1e4, size = 9, prob = p_samples$p)
2 ## distribution of predictions of G under our posterior
3 head(p_samples)
```

```
      p
1 0.6243390
2 0.8218246
3 0.3954844
4 0.5735221
5 0.7429660
6 0.6236375
```

```
1 head(RG_predictions)
```

```
[1] 5 8 2 4 7 5
```

Visualize the predictions

```
1 ggplot(data.frame(RG_predictions),  
2       aes(x = RG_predictions)) +  
3   geom_histogram() +  
4   scale_x_continuous(breaks=0:9) +  
5   labs(title = "Posterior predictions of G")
```

Two kinds of uncertainty

Posterior predictions propagate ALL modeled uncertainty

```
1 ggplot(data.frame(RG_predictions),  
2       aes(x = RG_predictions)) +  
3   geom_histogram() +  
4   scale_x_continuous(breaks=0:9) +  
5   labs(title = "Posterior predictions of G")
```

Prior Prediction

Prior prediction

We can also use simulation to better understand and select priors

```
1 ### our prior  
2 formula_list
```

```
[[1]]  
G ~ dbinom(R + G, p)
```

```
[[2]]  
p ~ dunif(0, 1)
```

```
1 prior_samples<-runif(1e4, 0, 1)  
2  
3 head(prior_samples)
```

```
[1] 0.08471689 0.46572544 0.69882652 0.36752163 0.18188801 0.62083441
```

Visualize the prior samples

```
1 ggplot(data.frame(prior_samples),  
2       aes(x = prior_samples)) +  
3   coord_cartesian(xlim = c(0, 1)) +  
4   geom_histogram(bins = 20)
```

Prior Predictive sampling

We can simulate data from this prior

```
1 prior_pred <- data.frame(p = prior_samples) |>  
2   mutate(G = rbinom(length(prior_samples), 9, p))  
3 head(prior_pred)
```

	p	G
1	0.08471689	2
2	0.46572544	3
3	0.69882652	5
4	0.36752163	4
5	0.18188801	2
6	0.62083441	7

Visualize

```
1 ggplot(prior_pred,  
2       aes(x = G)) +  
3   geom_histogram()
```

Linear regression

Why the Normal is so Normal

```
1 n_sims <- 1000
2 output<-data.frame(sums = rep(NA, n_sims))
3 for(i in 1:1000){
4   temp <- rbinom(1000, 20, 0.1)
5   output$sums[i] <- sum(temp)
6 }
7
8 ggplot(output, aes(x = sums)) + geom_density()
```

Looks familiar...

```
1 # convert to z score, x is for plotting
2 output <- output |>
3   mutate(z = scale(sums),
4          x = seq(-5, 5, length.out = n_sims))
5 ggplot(output,
6         aes(x = z)) +
7   geom_density() +
8   geom_line(aes(x = x, y = dnorm(x, 0, 1)), lty = 2)
```

Not just the binomial...

```
1 n_sims <- 1000
2 output<-data.frame(sums = rep(NA, n_sims))
3 for(i in 1:1000){
4   temp <- rpois(100, 3)
5   output$sums[i] <- sum(temp)
6 }
7
8 hist(temp)
```

Sums of random variables ~ N

```
1 ggplot(output, aes(x = sums)) + geom_density()
```

Again!

```
1 output <- output |>
2   mutate(z = scale(sums),
3          x = seq(-5, 5, length.out = n_sims))
4 ggplot(output,
5        aes(x = z)) +
6   geom_density() +
7   geom_line(aes(x = x, y = dnorm(x, 0, 1)), lty = 2)
```

Getting to know the Normal

- Normal (Gaussian) have two parameters: mean (μ) and standard deviation (σ)
- These two parameters describe the location and spread of the distribution

```
1 xrange<-seq(-7,7,length.out=100)
2
3 normals<-data.frame(x = rep(xrange, 4),
4                      y = c(dnorm(xrange, 0, 1),
5                            dnorm(xrange, 0, 1.5),
6                            dnorm(xrange, 0, 2),
7                            dnorm(xrange, 0, 2.5)),
8                      SD = rep(c("1", "1.5", "2", "2.5"), each = 100))
9
10 ggplot(normals, aes(x = x,y = y, color = SD)) +
11   geom_line()
```

```
1 xrange<-seq(-5,5,length.out=100)
2
3 normals<-data.frame(x = rep(xrange, 4),
4                      y = c(dnorm(xrange, 0, 1),
5                            dnorm(xrange, 0.5, 1),
6                            dnorm(xrange, 1, 1),
7                            dnorm(xrange, 1.5, 1)),
8                      Mu = rep(c("1", "1.5", "2", "2.5"), each = 100))
9
10 ggplot(normals, aes(x = x,y = y, color = Mu)) +
11   geom_line()
```

Fun fact: every normal can be a $N(0,1)$

```
1 xrange<-seq(-10,50,length.out=100)
2 wide_norm<-dnorm(xrange, mean = 20, sd = 12)
3 plot(xrange, wide_norm, type ="l",
4      main = "N(20, 12)")
```

Rescaling N(20,12)

```
1 wide_norm<-(wide_norm-20)/12
2 ## this is the same result as scale(wide_norm)
3 ## also called z-scores
4 plot(seq(-3,3,length.out=100), wide_norm, type="l",
5       main = "N(20, 12), rescaled to N(0,1)")
```

Model notation

Likelihood: $W \sim \text{Binomial}(N, p)$

Prior: $p \sim \text{Uniform}(0, 1)$

- Likelihoods put probability distributions on the data
- Priors put probability distributions on the parameters (!!!)

Building a model of human height

```
1 library(rethinking)
2 data("Howell1")
3 d<-Howell1
4 head(d)
```

	height	weight	age	male
1	151.765	47.82561	63	1
2	139.700	36.48581	63	0
3	136.525	31.86484	65	0
4	156.845	53.04191	41	1
5	145.415	41.27687	51	0
6	163.830	62.99259	35	1

Building a model of human height

$$h_i \sim \text{Normal}(\mu, \sigma)$$

What are the parameters in this model?

What is our next step?

Set priors!

How about Normal(0, 10)?

$$\mu \sim N(0, 10)$$

Thoughts?

```
1 prior_sims<-rnorm(1e4, 0, 10)
2 plot(density(prior_sims))
```

Rethink the prior for μ

- The average adult male is 176.5 cm
- the average adult female is 163 cm
- Maybe we should set $\mu = \frac{176.5+163}{2} = 169.75$

One more parameter needed

How certain are we that the true mean for this sample will be equal to 169.75?

- Completely certain? $\mu \sim N(169.75, 0.01)$
- Pretty darn sure? $\mu \sim N(169.75, 2)$
- Probably in the ballpark $\mu \sim N(169.75, 10)$
- OK, not too sure, but not a bad guess $\mu \sim N(169.75, 25)$

Simulate 4 priors for μ

```
1 all_the_priors <- data.frame(  
2   sigma1 = rnorm(1e4, 169.75, 1),  
3   sigma2 = rnorm(1e4, 169.75, 2),  
4   sigma10 = rnorm(1e4, 169.75, 10),  
5   sigma25 = rnorm(1e4, 168.75, 25))
```

Distributions of priors for μ

```
1 all_the_priors |>
2   pivot_longer(cols = everything()) |>
3   ggplot(aes(x = value * 0.0328084, color = name)) +
4   geom_density(alpha = 0.1) +
5   labs(x = "Height in feet", color = "Mu prior")
```

Now for σ

- How much do individuals vary on average from the population average? How variable is height?
- Recall $sd = \sqrt{\frac{\sum (x_i - \mu)^2}{n}}$
- The average deviation of an observation from the mean
- σ is constrained to be positive, our prior should reflect this

The impact of σ on our prior

Our prior for σ captures our beliefs about how much variance there is within the population.

How much variable is height across adults?

- No idea how variable the population is: $\sigma \sim \text{Uniform}(0, 50)$
- We know there's not much variation: $\sigma \sim \text{Uniform}(0, 10)$

Understanding the prior: prior predictive simulation

. Simulate μ

```
1 ### simulate mu prior: mu ~ N(169.75, 25)
2 mu_sims<-rnorm(1e4, 169.75, 25)
```

. Simulate σ

```
1 ### simulate sigma prior: sigma ~ U(0,50)
2 sigma_sims<-runif(1e4, 0, 50)
```

. Draw predictions from $N(\mu, \sigma)$

```
1 ### prior predictive distribution: y~N(mu, sigma)
2 height_preds<-rnorm(1e4, mu_sims, sigma_sims)
```

Simulating data from the prior

Is our prior reasonable?

```
1 data.frame(x = height_preds) |>  
2   ggplot(aes(x = x * 0.032)) + geom_density() + labs(caption = "height in feet")
```

Revise the prior!

Speaker notes